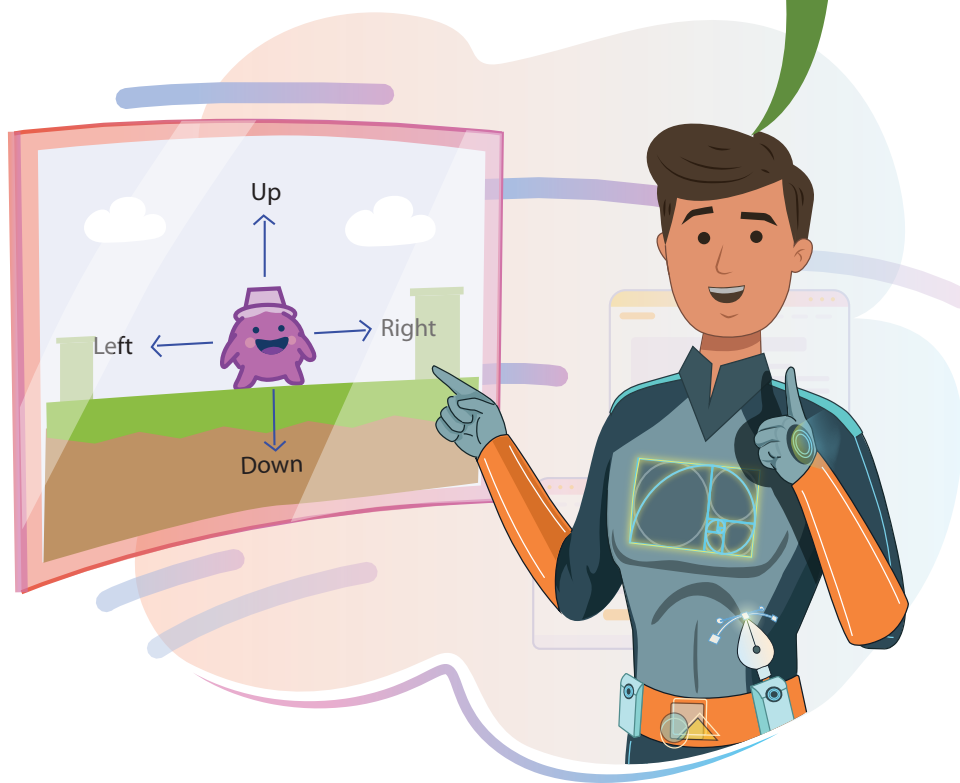


Movement in 3D Environment

11

The player drives a game. Depending on the game type, it interacts with and controls activities in a game. Movement is an essential activity in a game.



In the last lesson, we learnt more about video games and how they affect us, both positively and negatively. We explored the game design process and created a GDD for a litmus game. In this lesson, let's explore the game engine that we will use to create the game we've designed.

Unity: Unity is a cross-platform game engine developed by Unity Technologies. It is used to create three-dimensional (3D) and two-dimensional (2D) games, as well as interactive simulations and other experiences.



Installing Unity:

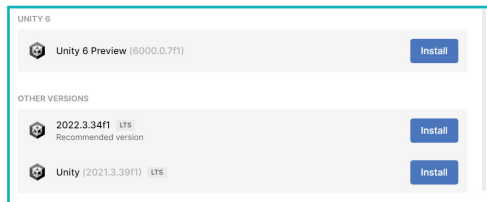
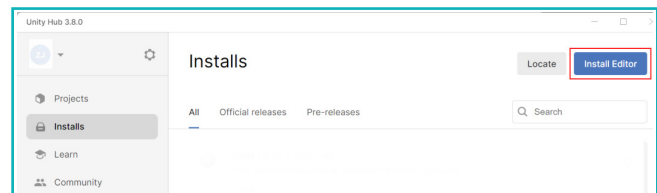
The Unity Hub allows us to install the Unity Editor, create projects, and manage our entire Unity experience. It provides a central location to manage Editor installations, accounts and licenses, and projects.

To install the Unity Hub for Windows, macOS, and Linux visit Download Unity on the Unity website:

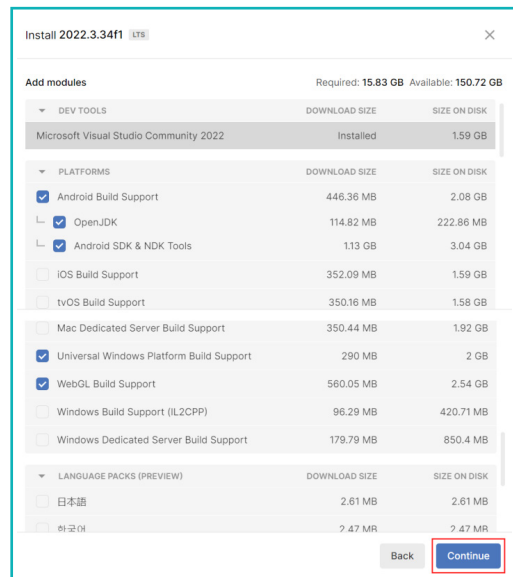
<https://bit.ly/3ntudbc>

To install the Editor:

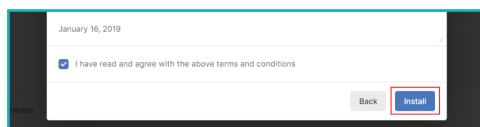
- 1 Go to the Installs tab.
- 2 Click on 'Install Editor' button. Select a version of the Editor as shown below.



- 3 Select the modules:
 - a. Android Build Support
 - b. Universal Windows Platform Build Support
 - c. WebGL Build Support

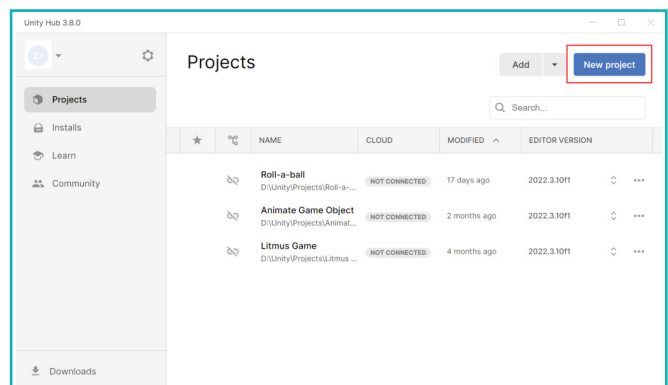


Once selected, click on 'Continue'.
Click on 'Install' in the next window.



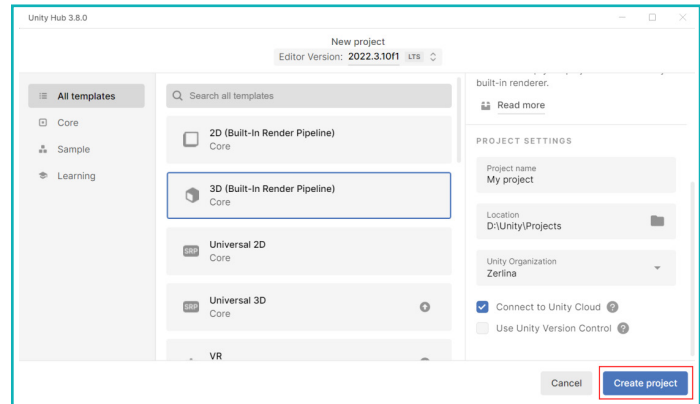
This will install the editor onto your system.

- 4 To start working on a project, go to the Projects tab.

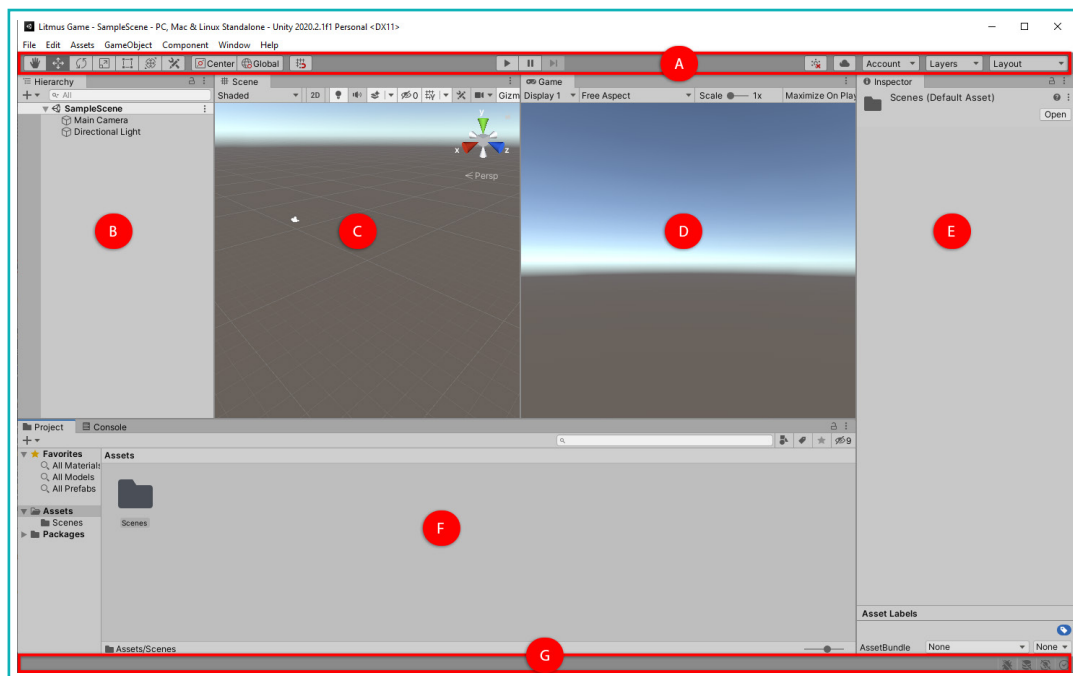


Unity stores projects on the computer's disk and in the cloud. We can open both, local and cloud projects. The projects tab also displays a list of created projects. To open a project that isn't on the list, click 'Add' and browse the computer for the project folder. To create a new project, click on 'New project'.

- 5 Select a project template, set a project name and location, and click on 'Create project'. We need a 3D template for the 3D game we are creating.



- 6 The Unity Editor will open. The main editor window is made up of tabbed windows which can be rearranged, grouped, detached and docked. The appearance of the editor can change depending on the project, developer, personal preference, and type of work. The default arrangement of windows gives us practical access to the most common windows.



- a. The **Toolbar** provides access to the most essential working features. Its left side contains basic tools to manipulate the Scene view and GameObjects within it.

In the centre are the play, pause and stop controls.

The buttons on the right give us access to Unity Collaborate, Unity Cloud Services and our Unity Account. These are followed by a Layer visibility menu and the Editor Layout menu (which provides alternate layouts for Editor windows and allows us to save our own custom layouts).

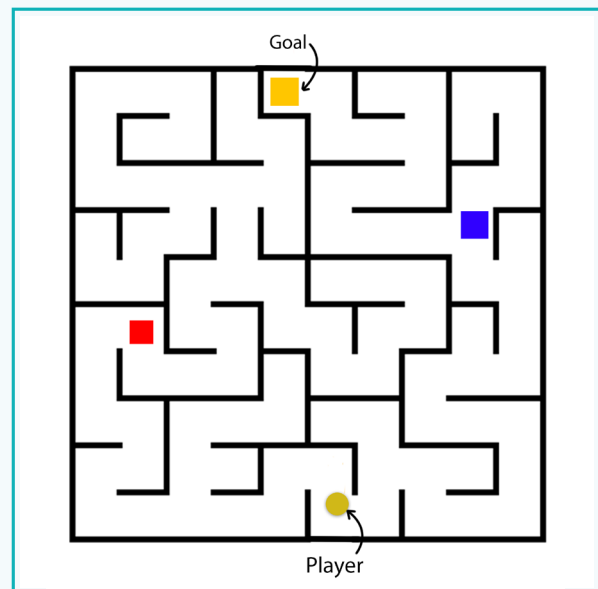
- b. The **Hierarchy** window is a hierarchical text representation of every GameObject in the Scene. It reveals the structure of GameObjects attached to each other.

- c. The **Scene** view allows us to visually navigate and edit a scene. It displays a 3D or 2D perspective, depending on the type of project.
- d. The **Game** view simulates what the final rendered game will look like through the Scene Cameras. When we click Play, the simulation begins.
- e. The **Inspector** window allows us to view and edit all the properties of the selected GameObject.
- f. The **Project** window displays a library of assets available to use in the project. When we import assets into our project, they appear here.
- g. The **Status** bar displays notifications about various Unity processes and quick access to related tools and settings.

Activity

■ Create a part of the litmus game that you have designed in the earlier lesson.

As per our GDD, we need to create a player that moves around a 3D maze.



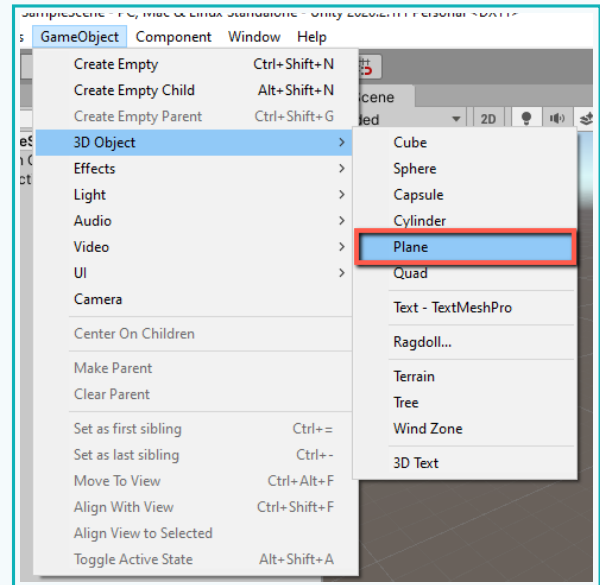
To start with, we will create a ground plane where the player will move around.

Every object in our game is a **GameObject**, from characters and collectible items to lights, cameras and special effects. A GameObject is the fundamental object in Unity that represents characters, props and scenery.

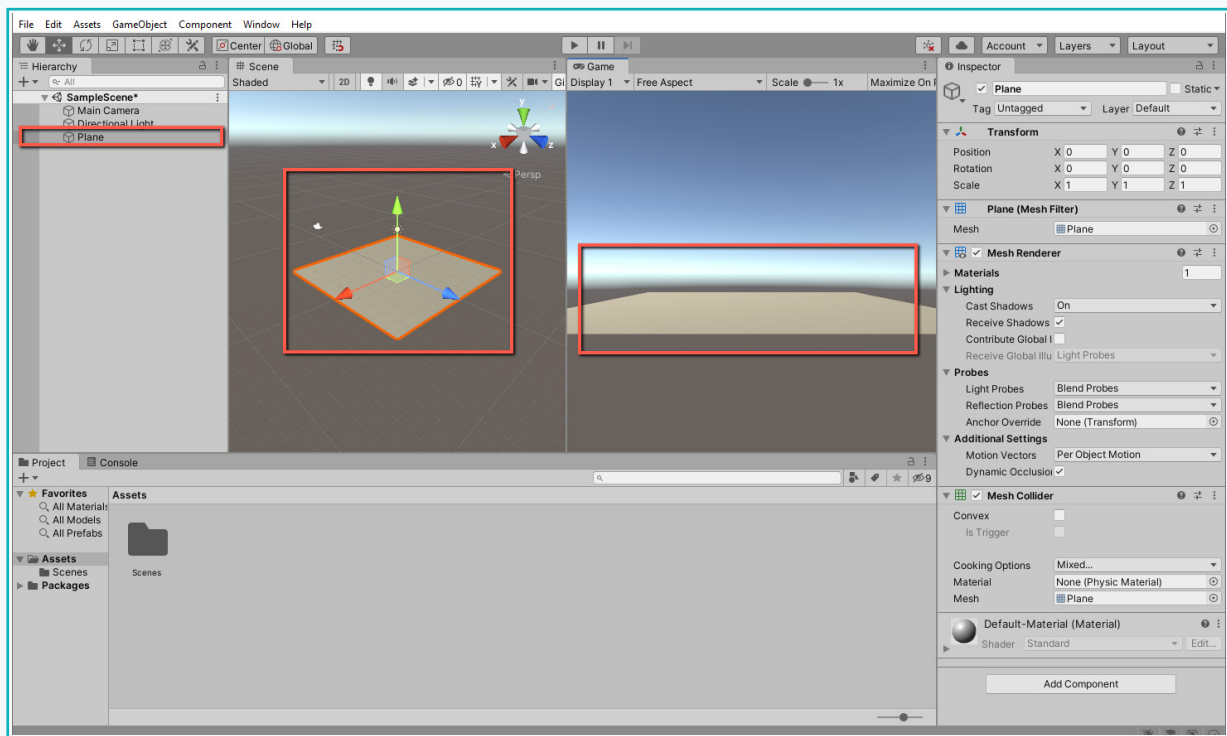
Activity

- 1 To create the ground, use a 3D GameObject plane

We can create a GameObject in multiple ways. One of them is by selecting a 3D Object - plane in the GameObject menu.

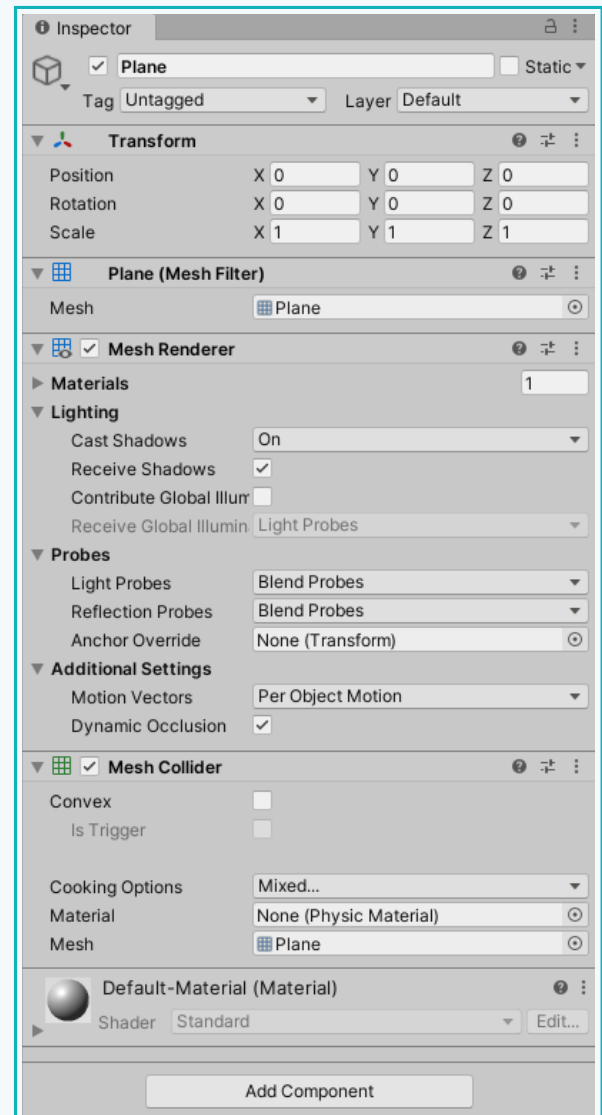


Once we select the plane object, it will appear in the scene window, game window and hierarchy window.



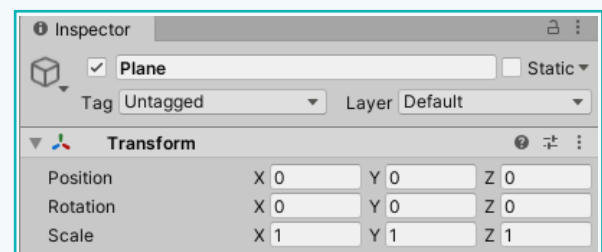
Activity

When we select the GameObject, the inspector panel displays its various properties.



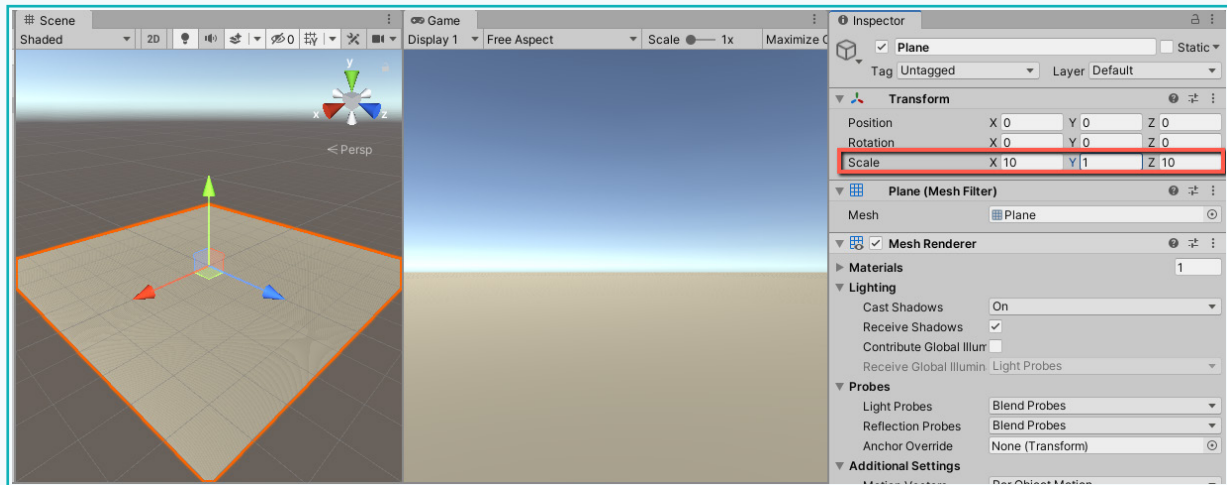
A GameObject cannot do anything on its own, we need to assign properties before it can become a character, an environment, or a special effect. GameObjects do not accomplish much in themselves, but they act as containers for components. The Inspector window displays the components assigned to the GameObject.

A GameObject always has a Transform component attached (to represent position, orientation, and scale) and it is not possible to remove this. This component determines the position, rotation and scale of each object in the scene.

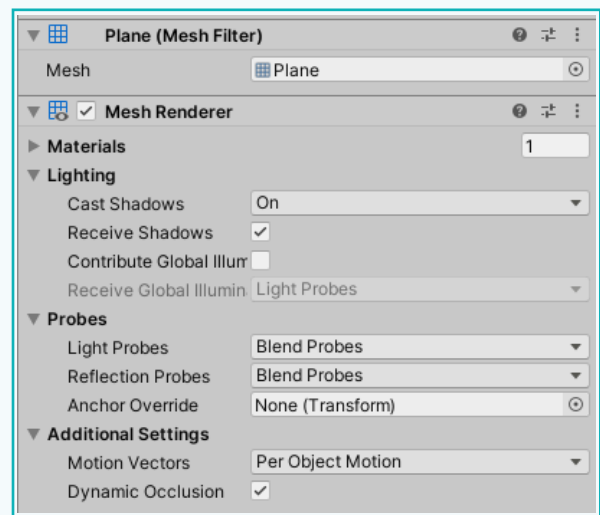


Activity

Observe the change as we scale the object. We need to scale it to provide a larger surface area for the player to move around in the maze.



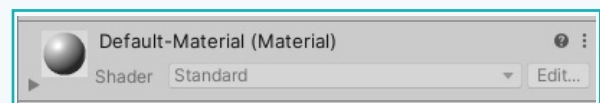
The object also has a mesh filter and mesh renderer component to draw the object surface.



A **Mesh** is the main graphics primitive of Unity. Meshes define the shape of an object.

Materials define how a surface should be rendered by including references to textures tiling information, colour tints and more. The available options for a material depend on the shader used.

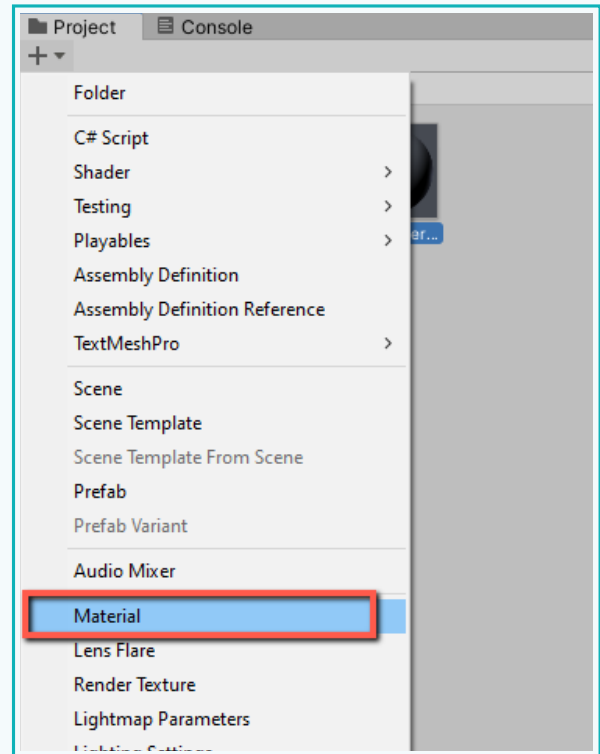
Presently, the plane object is assigned a default material.



We can change the appearance of the object by assigning a new material.

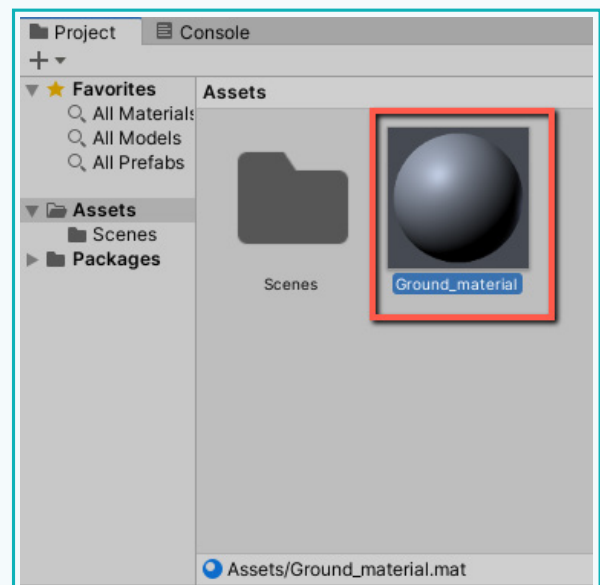
Activity

- 2 To create a new material, go to Assets >> Create >> Material from the project view context menu



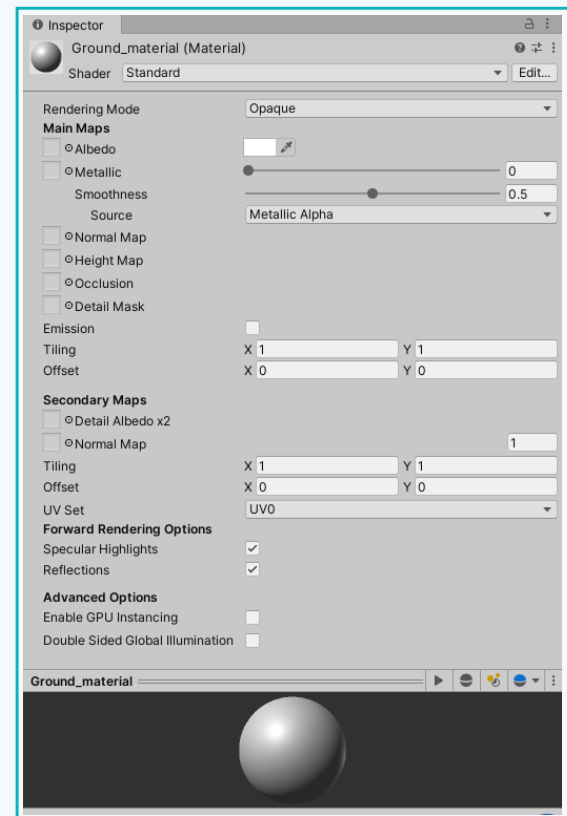
It is good practice to name all the assets we create.

- 3 Name the material "Ground_material"



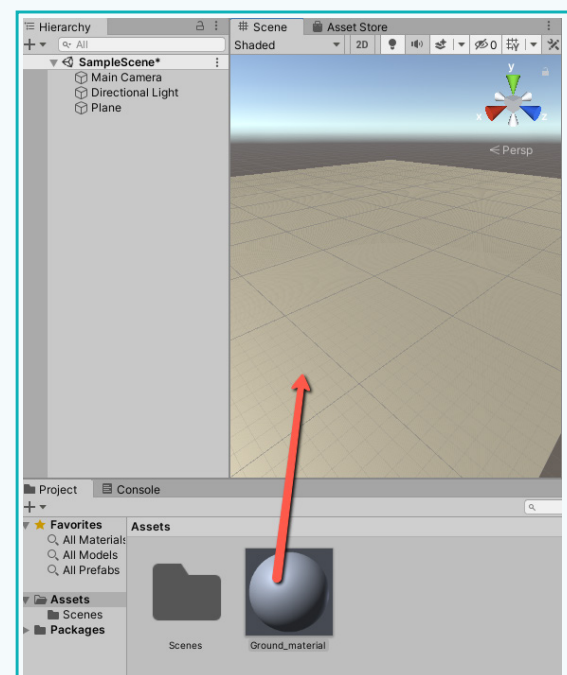
Activity

By default, new materials are assigned the standard shader with all map properties empty:



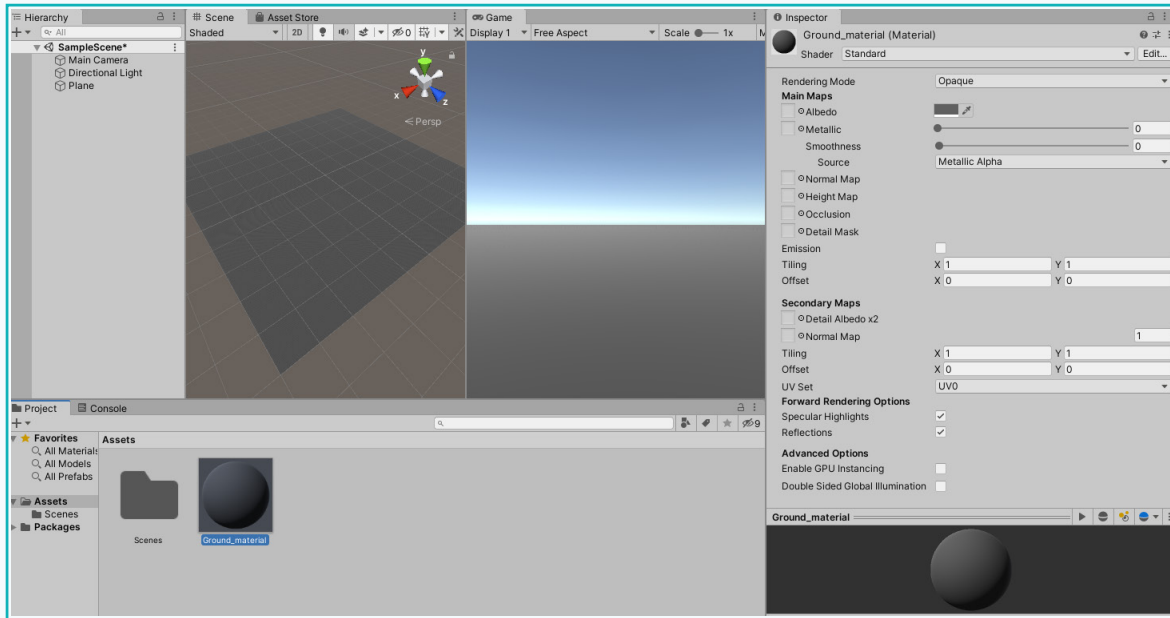
Once the material is created, we can apply it to an object and tweak all of its properties in the Inspector.

- 4 To apply the material, drag it over the GameObject



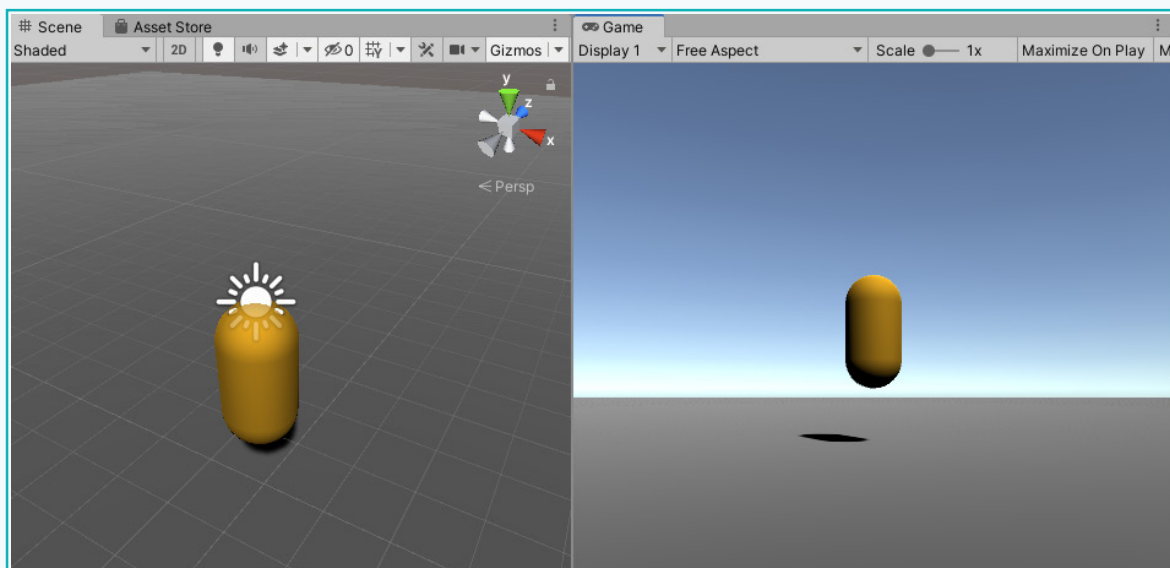
Activity

- 5 To change the object's colour, change the Albedo colour value



Follow the same method to create the player.

- 6 Use a capsule object to create the player



Activity

Note:

Since we are working in a 3D workspace, it is possible for us to view, position and orient the object from different perspectives. The scene view is our interactive view into the world we are creating. We can use it to select and position scenery, characters, cameras, lights, and all other types of GameObjects. When we begin working with Unity, we need to first learn to select, manipulate, and modify GameObjects in the scene view.

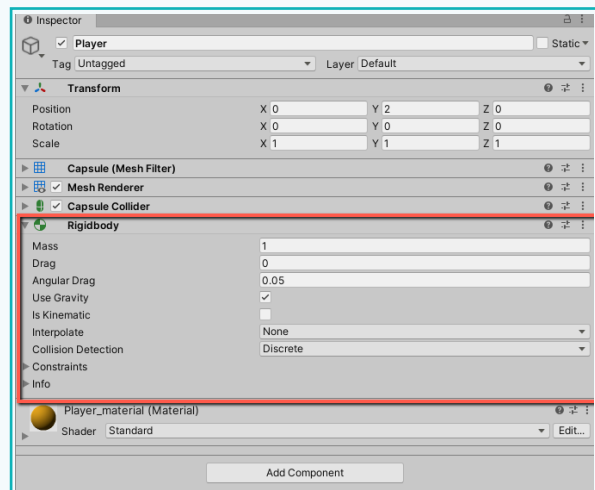
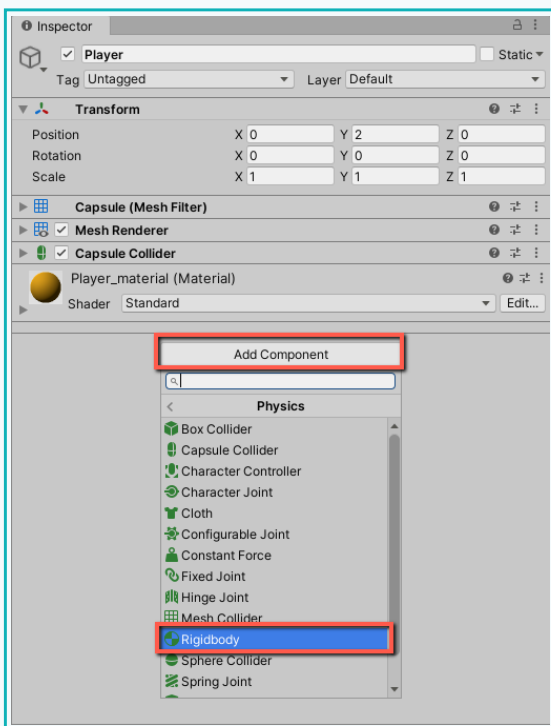
Move	When the hand tool is selected (shortcut: Q), click and drag to move the camera around.	      
Orbit	Hold Alt (Windows) or Option (macOS), left click and drag to orbit the camera around the current pivot point.	      
Zoom	Hold Alt (Windows) or Option (macOS), right-click and drag to zoom into the scene view.	      

To make the player move around the surface, we need to ensure that the player has physical properties like mass and gravity and that it reacts with other physical objects. For this, we need to add a Rigidbody component to the player.

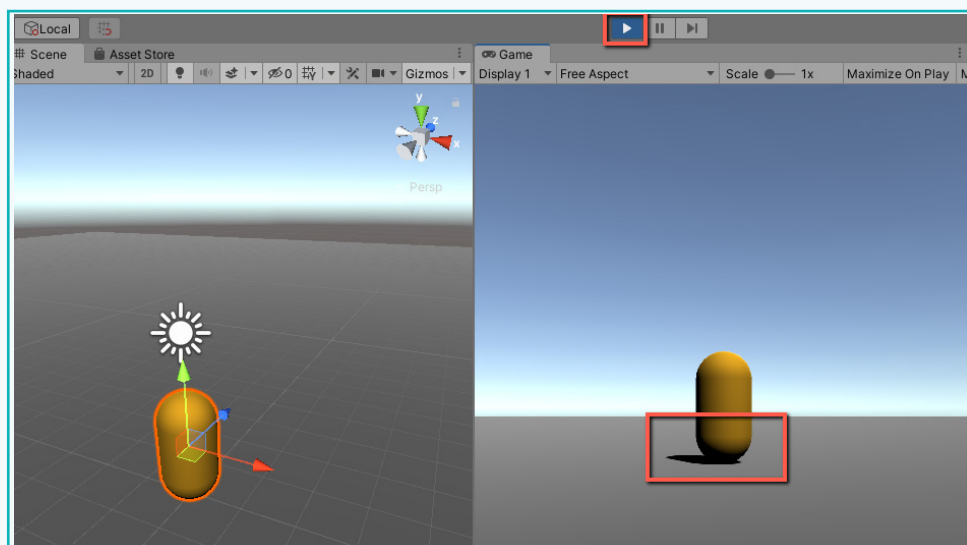
GameObjects can be affected by gravity and function under additional forces. They can also communicate with other objects through the NVIDIA PhysX physics engine. For any of these functions, a Rigidbody is required. Rigidbodies allow GameObjects to function as per the properties of physics.

Activity

- 7 Select the player GameObject. In the inspector panel, click on “Add Component” and select Rigidbody from the Physics category. Set Rigidbody values as per the image.



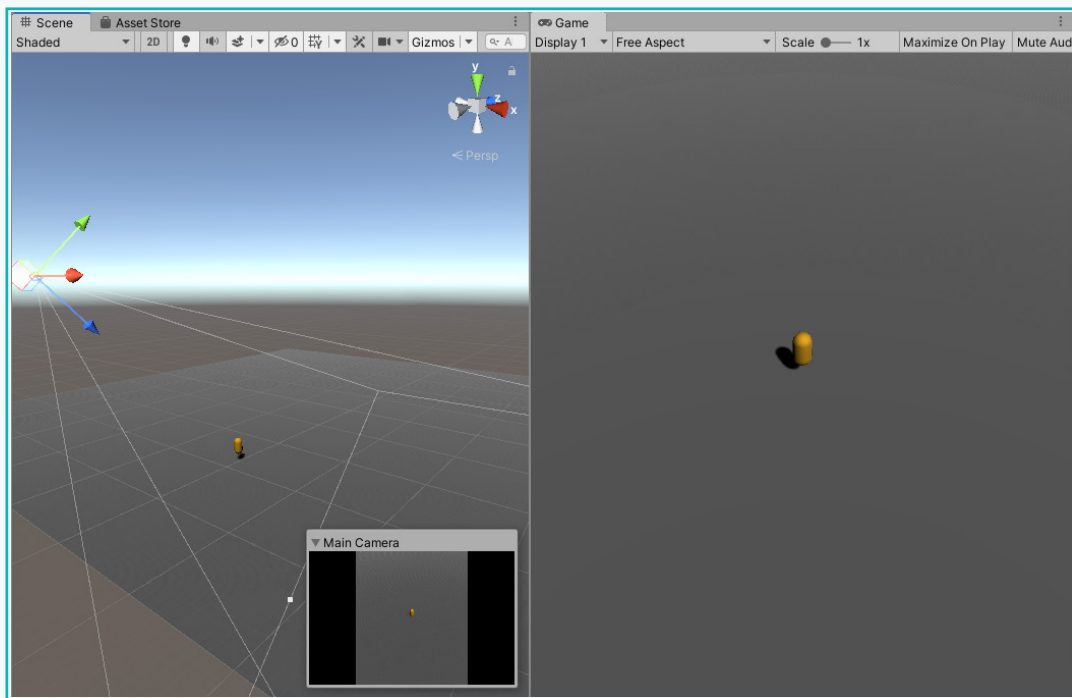
- 8 Press play to test whether the player is reacting correctly



Activity

The player should fall and rest on the ground plane.

We can also adjust the camera position and angle to change the game view.



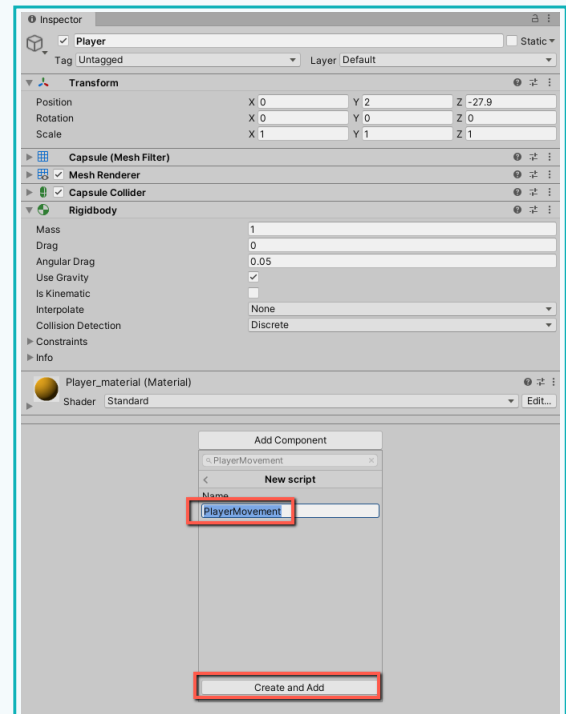
Next, let's write a script to move the player. Scripting is an integral component in all Unity applications. Most applications require scripts to respond to player feedback. Scripts also ensure that events in the game occur when they should. Scripts can be used to generate graphical effects, monitor the physical behaviour of objects, and implement a custom AI framework for game characters.

Scripts are also called behavioural components since they define how a GameObject behaves in a game. Unity's native programming language is C#.

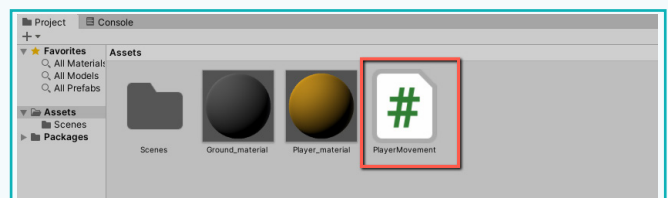
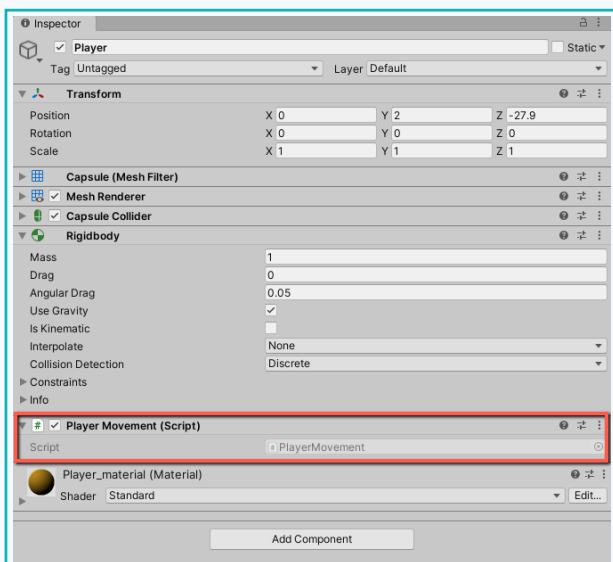
Activity

9 Create a script for the player

Select the player, add a New Script component and name the script.

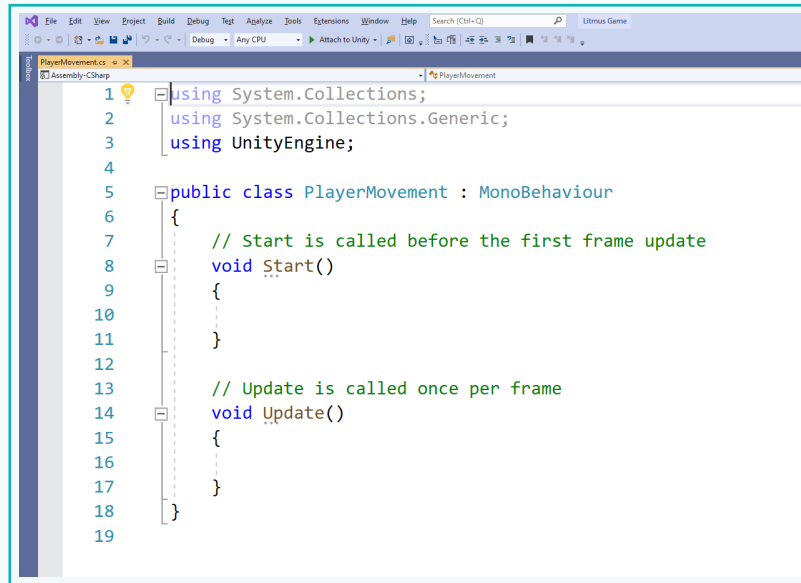


Observe that the script appears in the inspector window as well as the project window.



Activity

- 10 Double click on the script to open it in the Integrated Development Environment (IDE)



Unity provides a basic structure for the script.

A script makes its connection with the internal workings of Unity by implementing a class which is derived from the built-in class called MonoBehaviour. The class name is picked up from the file name we supply while creating it. The class name and file name must be the same to enable a script to attach to a GameObject.

By default, there are two functions defined inside a class:

- void Start(): The Start function is called by Unity before the gameplay begins.
- void Update(): The Update function holds code that will handle the frame update for the GameObject. Update is called on every frame, multiple times each second!

Bytes

An Integrated Development Environment (IDE) is a computer software that provides tools and facilities to develop other pieces of software. An IDE makes the development process easier.

Unity supports the following IDEs:

- Visual Studio (default IDE for Windows and MacOS)
- Visual Studio Code
- JetBrains Rider

Activity

A namespace is a collection of classes. These are referred to using a chosen prefix with the class name. Their purpose is to help organize your scripts and to avoid conflicts between scripts.

At the top of C# scripts in Unity, by default, we see two lines:

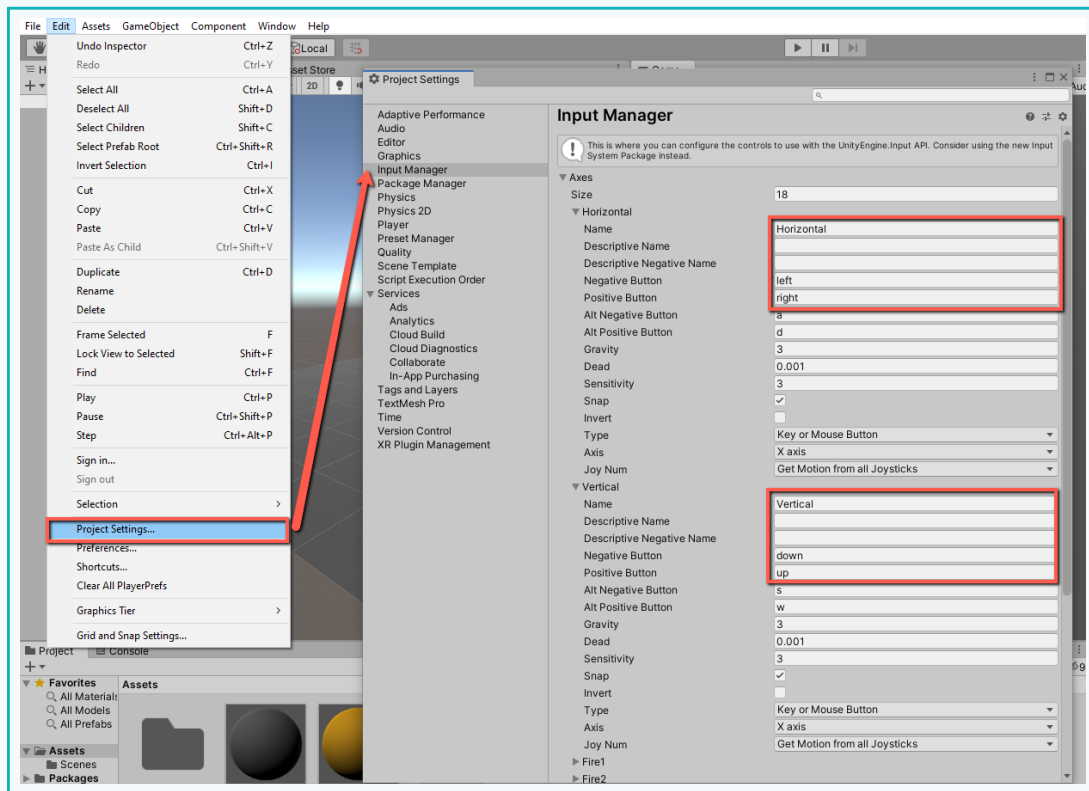
- using UnityEngine;
- using System.Collections;

UnityEngine and System.Collections are both namespaces. The “using” keyword means that anything within the namespace that follows it can be used in the script.

Curly braces: Used to group a set of commands or statements. An opening curly brace { must always be followed by a closing curly brace }, with the command in between.

Before we edit the script, let's understand the kind of input that we need for this functionality. We need to provide player movement over the ground plane—along X and Z axes. For any game executed on a computer, the input is mostly accepted by keyboard and mouse. Here, we will consider the keyboard.

If we observe the input settings, arrow keys are used for movement. They use the keywords “Horizontal” and “Vertical”.



Activity

11 Edit the script

```

1  using System.Collections;
2  using System.Collections.Generic;
3  using UnityEngine;
4
5  public class PlayerMovement : MonoBehaviour
6  {
7      private Rigidbody rb;
8      float speed = 10f;
9
10     void Start()
11     {
12         rb = GetComponent<Rigidbody>();
13     }
14
15     void Update()
16     {
17         float moveX = Input.GetAxis("Horizontal");
18         float moveZ = Input.GetAxis("Vertical");
19         Vector3 move = new Vector3(moveX, 0, moveZ);
20         rb.AddForce(move * speed);
21     }
22 }
    
```

```
private Rigidbody rb;
```

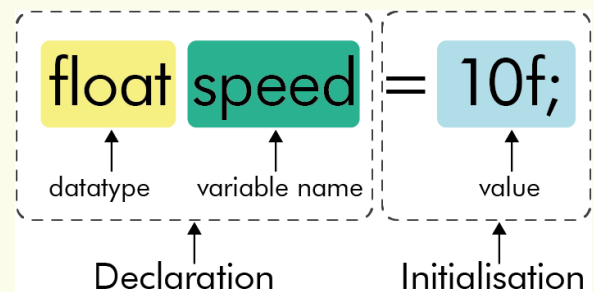
Creates a variable of type Rigidbody

```
float speed = 10f;
```

Creates a variable of type 'float' with a value of 10

Recall:

Variables: variables are containers for information.



Activity

Different types of containers—called datatypes—are used for different types of information. Commonly used datatypes:

Int	Stores 32 bit integer numbers, which are positive or negative whole numbers.
Float	Stores floating point numbers. The float datatype is the default type of number in Unity.
Bool	Short for Boolean, this data type stores true or false values.
Char	Stores a single character such as a number, letter, space, or special character. A char value should be written in single quotes.
String	Stores letters, numbers, and other special characters in the form of words or sentences. A string value should be written inside double quotes.

When the game starts, we access the Rigidbody component of the GameObjects. This allows us to assign movement during the gameplay.

```
void Start()
{
    rb = GetComponent<Rigidbody>();
}
```

We want to identify the axis along which the player commands movement during runtime. We require movement along X and Z axes. We will create two variables to store input received from the player. Use these variables in the update function.

```
float movex = Input.GetAxis("Horizontal");
float movez = Input.GetAxis("Vertical");
```

We shall use the values of these variable to determine the GameObject's location in 3D space.

Vector3 datatype is a representation of 3D vectors and points. This structure is used throughout Unity to pass around 3D positions and directions.

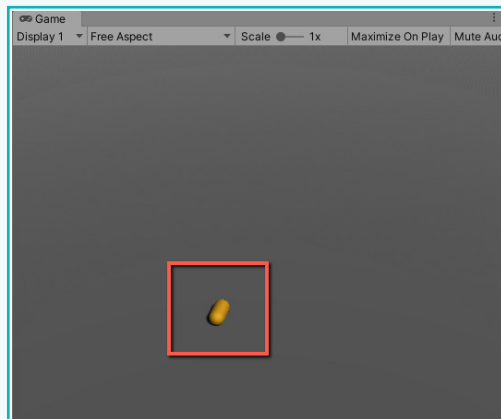
```
Vector3 move = new Vector3(movex, 0, movez);
```

To make the Rigidbody move, a force needs to be applied in a particular direction. This direction is received through the previous statement. It may change at any time during the gameplay as the user presses keys. Hence, this statement is executed in the update function. We need to obtain the force required in a particular direction and speed. We will use the following statement:

```
rb.AddForce(move * speed);
```

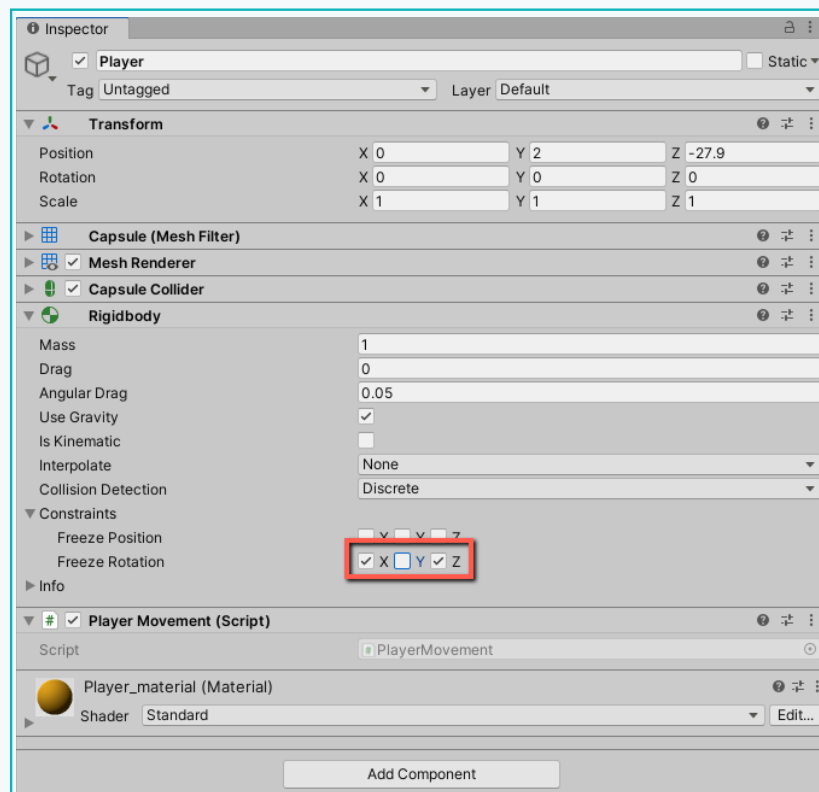
Activity

Once the script is complete, save it and return to Unity. Use the test button to check if the player movement is functional. In the game window, press the arrow keys to test the movement.



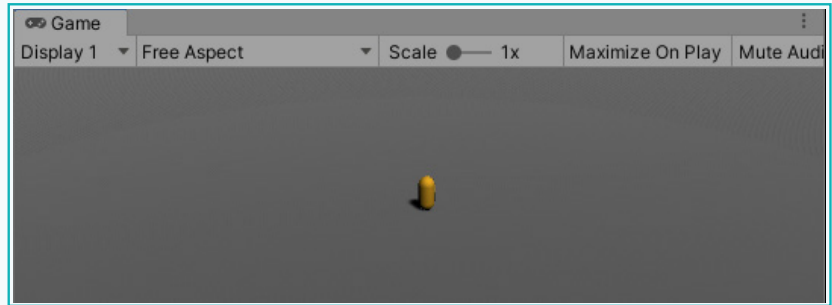
Observe that the player moves, but it tumbles and falls over. To fix these, we will add a few constraints.

- 12 Add constraints to the player rotation as shown in the image



Activity

- 13 Save and test again



The player is now moving the way we intended!
This completes the player movement.

Exercise

Fill in the blanks

- 1 _____ contains basic tools for manipulating the scene view and GameObjects within it.
- 2 The _____ is a hierarchical text representation of every GameObject in the scene.
- 3 The _____ simulates what our final rendered game will look like through our scene cameras.
- 4 GameObjects do not accomplish much in themselves but they act as containers for _____.
- 5 The _____ component determines the position, rotation, and scale of each object in the scene.
- 6 Scripts are also termed as _____ components, since they define how a GameObject behaves in the game.

In this lesson, we learnt about Unity as a cross-platform game engine used to create 3D and 2D games. We explored it and created a part of the Litmus game.

Gravity, force, and speed are all important when we think about movement — in real life and games. These factors also influence free fall. Read up more about this concept!



Tech Flash

- **AddForce** : Applies a force of a vector to the Rigidbody.
- **Start()** : Called by Unity before gameplay begins.
- **Update()** : Called on every frame, multiple times per second.